

# LLM-Assisted Assertion Generation

## with Formal Verification

Problem Set 3 · Formal Methods in Hardware Verification

---

Presented by:

**N. Dona Priyamvada (230101069)**

**Shreesh Savlani (230101096)**

**Udayraj Choudhary (230101107)**

**Wagh Rutvij Ganesh (230101113)**

### Scope at a Glance

6

RTL Designs

96

Total Assertions

3C

Framework

FV

Formal Validation

# Problem Statement · PS3

## The Challenge

- Writing high-quality SVA assertions is time-consuming and error-prone
- Requires deep design understanding — unscaleable manually
- LLM-generated assertions may be incomplete, redundant, or unsound

## Project Goal

- Automatically generate candidate assertions using an LLM from RTL/specs
- Formalize assertions as SVA — classify by type (safety, liveness, invariant)
- Validate assertions with formal verification tools (SymbiYosys / BMC)
- Analyze correctness, usefulness, and limitations — Three C's framework

**KEY SEPARATION — LLM handles Generation · Formal Methods handle Validation**

# Motivation



## Scale Problem

Modern RTL designs have thousands of signals. Manual assertion writing doesn't scale.



## LLMs Know RTL

LLMs trained on code understand Verilog semantics and can propose meaningful properties.



## LLMs Aren't Oracles

LLM assertions can be vacuous, over-restrictive, or simply wrong — formal verification is the safety net.



## Hybrid Power

LLM creativity + formal rigor = faster, more trustworthy assertion suites.

# Research Foundation · LAAG-RV & Related Work

## LAAG-RV · LLM Assisted Assertion Generation for RTL Verification [IIT Guwahati, ITC India 2024]

### Evaluation Framework — Three C's:

Correctness

Consistency

Completeness

01

#### Initial Domain Cognition

Feed domain knowledge + block diagram to LLM

02

#### Design-Specific Prompt

NL spec + architecture given as starting prompt

03

#### One-Time Verilog Loop

RTL fed once to sync signal names in SVA

04

#### Simulation Validation

VCS logs → LLM re-prompted to fix errors manually

### Key Findings from LAAG-RV

LLMs generate more assertions than OpenTitan provides — covering missed design aspects

One-time Verilog loop reduces signal-name errors, needing fewer re-prompts than ChIRAAG

Simulation-based validation catches syntax & timing errors — but lacks formal (exhaustive) guarantees

Three C's framework (Correctness / Consistency / Completeness) adopted as our evaluation standard

## ChIRAAG [ISVLSI 2024 · IIT Guwahati]

Formats spec as structured JSON → feeds to GPT-4 → generates SVA → auto-feeds simulation logs back to LLM for error correction.

✗ JSON formatting is manual overhead.

✗ More prompts needed vs LAAG-RV.

✓ 27% raw error rate; fully automated feedback loop.

## SANGAM [ICLAD 2025 · IIT Guwahati]

Uses Monte Carlo Tree Search (MCTS) + DeepSeek-R1 to iteratively reason about assertions across multiple paths. Multi-modal: uses text, waveforms, architecture diagrams.

✗ High computational cost — 20 API calls per signal.

✗ Complex for a baseline study.

✓ Generates 2× more assertions; >90% coverage.

## Why LAAG-RV for our project?

→ Transparent, reproducible pipeline — no black-box MCTS overhead

→ Introduced the Three C's framework we directly adopted

→ One-time Verilog loop matches our signal-sync step exactly

→ Our work EXTENDS it: replaces simulation-only with SymbiYosys formal verification

LAAG-RV = our baseline · ChIRAAG = parallel approach (more prompts, manual JSON) · SANGAM = state-of-the-art (too complex for baseline) · Our addition: Formal Verification layer

# Background · LLMs + Formal Verification

## Large Language Models

- ▶ Generative models fine-tuned on code corpora (Verilog, SV)
- ▶ Prompted with RTL or natural-language spec to produce SVA
- ▶ Output is probabilistic — needs external validation
- ▶ Models used: ChatGPT 5.3 (cloud LLM APIs)

## Formal Verification

- ▶ Mathematically proves or disproves properties over ALL inputs
- ▶ SVA (SystemVerilog Assertions) — safety, liveness, invariants
- ▶ SymbiYosys: open-source formal verification toolchain
- ▶ BMC (Bounded Model Checking) + unbounded PROVE mode

### SVA Property Types →



**Safety:** Bad thing never happens



**Liveness:** Good thing eventually happens



**Invariant:** Always holds (structural)



# Research Gap

## No Systematic Quality Evaluation

Prior work generates LLM assertions but lacks rigorous frameworks to classify correctness, consistency, and completeness.

## Assertion Soundness Blindspot

LLMs produce assertions that "look right" syntactically but are semantically incorrect (vacuous disable iff, wrong timing, over-restrictive guards).

## Combinational vs Sequential Confusion

LLMs mix clocked SVA idioms with combinational DUTs — causing false failures that look like design bugs.

## Missing Counterexample Feedback Loop

Most tools generate assertions once; no refinement loop feeds formal counterexamples back to the LLM for correction.

# Proposed Approach · System Overview

## 01 LLM Assertion Generation

- ✓ RTL designs as LLM input
- ✓ Prompt-engineered queries
- ✓ Raw assertions extracted & deduplicated

## 02 Assertion Formalization

- ✓ Clean syntax → valid SVA
- ✓ Classify: safety / liveness / invariant
- ✓ Map to design intent & add clocking

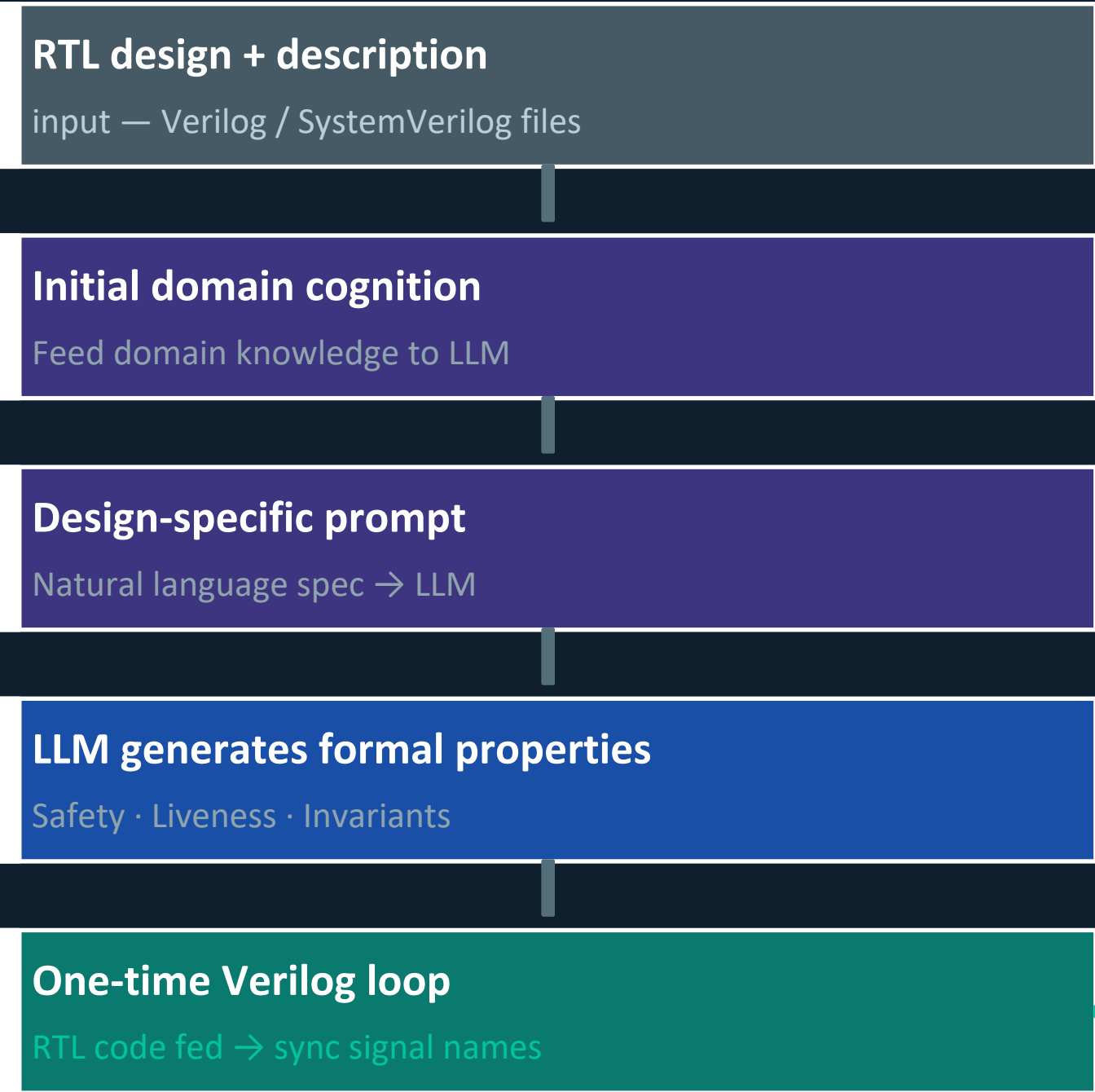
## 03 Formal Verification

- ✓ SymbiYosys BMC + PROVE modes
- ✓ Pass/Fail per assertion recorded
- ✓ Counterexamples analysed

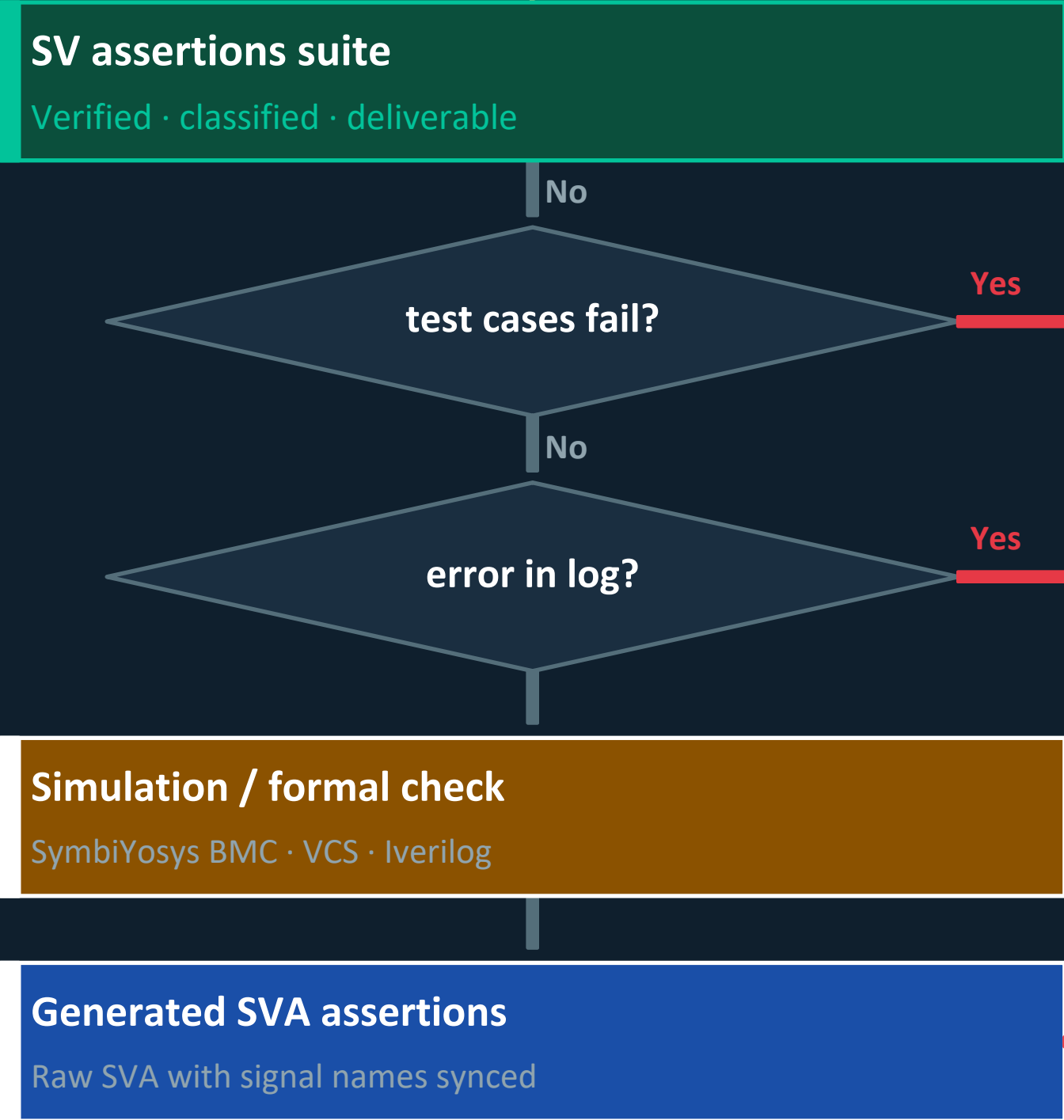
## 04 Critical Analysis · Three C's Framework: Correctness · Consistency · Completeness

# System Pipeline · LLM-Assisted Assertion Generation Workflow

## LLM Generation Phase



signals synced



- LLM stages
- Key steps
- Checks
- Final output
- SVA flow



# LLM-Based Assertion Generation

## Prompting Strategy

- "Generate safety assertions for this Verilog module"
- "What invariants must always hold for this FSM?"
- "Identify any temporal properties for the counter design"
- "Generate assertions for all state transitions"

⚠ LLMs not treated as oracles — all assertions sent to formal validation

## Assertions Generated per Design

|                 |    |
|-----------------|----|
| Full Adder      | 12 |
| FSM Controller  | 25 |
| Up/Down Counter | 8  |
| 2:1 Mux         | 15 |
| Padder Design   | 20 |
| Eq. Resolver    | 16 |

Total: 96

# Assertion Formalization

## Syntax Cleaning

1

Remove LLM prose, fix typos, convert to valid SVA property syntax with correct keywords

## Clock & Reset Binding

2

Add @(posedge clk) sampling, disable iff (!rst\_n) guards, and \$past() references with correct cycle offsets

## Type Classification

3

Label each assertion: Safety ( $| \rightarrow$ ), Liveness ( $| \rightarrow \#[1:N]$ ), Invariant (always), Temporal sequence

## Intent Mapping

4

Match each assertion to its design intent — state transition, output correctness, data path, gating

Example — Full Adder Carry (from raw text to formal SVA):

LLM: "carry should be 1 when at least two inputs are 1"  $\rightarrow$  SVA:  $((i_a + i_b + i_{cin}) \geq 2) \rightarrow (o_{carry} == 1);$

# Tool & Assertion Challenges

## Synchronous Reset Timing

Timing Bug

LLMs write reset checks as same-cycle properties ( $| \rightarrow$ ). For synchronous resets, the value updates AFTER the clock edge — must use next-cycle ( $| \Rightarrow$ ). Counter A4 failed this.

## Combinational DUT + Clocked SVA

False Fail

Mux is purely combinational. LLM applied  $\$changed()$  and  $@(posedge\ clk)$  idioms. SVA checks sampled at clock boundaries can give false failures. A12–A14 were over-restrictive.

## Vacuous Disable iff

Vacuous Pass

FSM A2: disable iff ( $!rst\_n$ ) was used in a property that checks reset itself — the property is disabled exactly when the interesting event occurs. PASS  $\neq$  correct property.

## Over-Restrictive Liveness

Spec Flaw

Full Adder A10: asserts that any input change must change outputs. For a full adder,  $0-1-1 \rightarrow 1-0-1$  keeps outputs identical — LLM missed this Boolean non-monotonicity.

# Formal Verification Setup

## Toolchain

### SymbiYosys

Open-source formal verification front-end

### Yosys

RTL synthesis + elaboration

### Boolector / Bitwuzla

SMT solver backend

### BMC Mode

Bounded Model Checking — finds counterexamples

### PROVE Mode

Unbounded proof — exhaustive verification

### Cover Property

Reachability check — verifies a target state is reachable

## RTL Designs Verified

### Full Adder (1-bit)

12 assertions

Combinational

### FSM Controller

25 assertions

Sequential

### Up/Down Counter

8 assertions

Sequential

### 2:1 Mux

15 assertions

Combinational

### Padder Design

20 assertions

Sequential

### Equivalence Resolver

16 assertions

Sequential

*Verification Rule: Every assertion — regardless of how obvious — was formally checked. No assertion was assumed correct by inspection.*

# Critical Analysis

- Which assertions are correct but trivial?
- Which assertions are incorrect or over-restrictive?
- Which assertions reveal bugs or corner cases?

## What Does this mean?

- **Correct but trivial:**

Assertions that are true but too simple or redundant, so they don't meaningfully verify the design.

- **Incorrect / over-restrictive:**

Assertions that wrongly constrain behavior or have timing/logical errors, rejecting valid design operation.

- **Bug-revealing / corner-case:**

Assertions that target edge cases and critical logic, helping uncover real bugs or hidden issues.



# Results · Three C's Evaluation

| Design          | Assertions | Correctness | Consistency | Completeness | Key Issue                           |
|-----------------|------------|-------------|-------------|--------------|-------------------------------------|
| Full Adder      | 12         | 91.7%       | 100%        | 85%          | A10: over-restrictive input-change  |
| FSM Controller  | 25         | 88%         | 100%        | 83%          | A2: vacuous reset; A23: liveness    |
| Up/Down Counter | 8          | 87.5%       | 100%        | 83%          | A4: wrong sync reset timing         |
| 2:1 Mux         | 15         | 73%         | 100%        | 92%          | A12-14: clocked check on comb. DUT  |
| Padder          | 20         | 85%         | 100%        | 70%          | A16: env-dependent; A20: too strict |
| Eq. Resolver    | 16         | 93.75%      | 100%        | 70%          | A16: liveness not guaranteed by RTL |

Consistency = 100% across ALL designs · No DUT bug revealed — issues were assertion-quality problems, not RTL bugs

# Critical Analysis • Key Findings

## ✗ Incorrect / Over-Restrictive

- > Full Adder A10: input change  $\rightarrow$ /output change (Boolean non-monotonicity missed)
- > FSM A2: disable iff (!rst\_n) disables the very property it's supposed to prove
- > Counter A4: same-cycle check on synchronous reset (must use  $\mid\Rightarrow$ )

## ⚡ Corner-Case Oriented

- > FSM A23: liveness — eventual return to IDLE — not guaranteed without input assumptions
- > Padder A16: eventual buffer\_full depends on environment, not pure DUT
- > Resolver A16: EOC liveness similarly environment-dependent

## ✓ Strongest Assertions

- > Full Adder A8, A9: end-to-end sum and carry equations — strongest FV checks
- > Full Adder A4–A7: internal half-adder stage decomposition (white-box)
- > FSM A3: state updates only from next-state register (catches hidden corruption)

*Final Verdict: LLMs produce non-trivial, mostly correct assertions — but formal verification catches 10-27% incorrect properties per design*

# Comparison • LLM vs Formal vs Combined

| Dimension      | LLM-Only        | Formal-Only           | Combined (Our Approach)        |
|----------------|-----------------|-----------------------|--------------------------------|
| Speed          | ⚡ Very fast     | 🐢 Slow for complex    | ⚡ Fast generation, targeted FV |
| Soundness      | ❌ No guarantee  | ✅ Mathematical proof  | ✅ Proven via FV backend        |
| Coverage       | High (creative) | Limited (manual spec) | High + validated               |
| Incorrect Rate | 10–27% flawed   | N/A (proven)          | Caught by FV pass/fail         |
| Scalability    | Very high       | Moderate              | High (LLM scales, FV targets)  |
| Bug Reveal     | Possible (weak) | Strong if complete    | Strong (formal validates)      |
| Completeness   | 70–92% (varies) | Depends on engineer   | 70–92% with quality check      |

The combined approach delivers LLM speed and creativity with formal soundness — neither alone achieves both.



# Conclusion

[01] LLMs generate non-trivial, semantically meaningful assertions — not just syntactic noise

[02] Formal verification is non-negotiable — it caught incorrect and vacuous assertions that passed superficial review

[03] Consistency = 100% across all designs; Correctness 73–94%; Completeness 70–92%

[04] Three systematic LLM failure modes: timing errors, comb vs seq confusion, Boolean non-monotonicity

[05] Counterexample-driven refinement loop is the natural next step — LLM + FV as a co-operative system

**LLMs write the first draft. Formal verification signs it. Together, they scale hardware verification.**